

DAY 07

JavaScript Masterclass

Array Methods Part 2

Deep Dive: `forEach` • `map` • `filter` • `reduce`

`</>` Higher-Order Functions in Modern JavaScript & React

Session Agenda



1. `forEach()`

Iterate elements cleanly without creating returned arrays.



2. `map()`

Transform values into a brand new array of identical length.



3. `filter()`

Select matching items based on custom boolean conditions.



4. `reduce()`

Accumulate entire array into a single combined output value.

MOTIVATION

Why Do We Need These Methods?

× traditional for-loop

```
let numbers = [1, 2, 3, 4, 5]; let doubled = []; for (let i = 0; i < numbers.length; i++) { doubled.push(numbers[i] * 2); } console
```

The Golden Rule: Which Method to Choose?

forEach()

"Just Do It"

Execute side-effects for each item (log, DOM update). Returns **undefined**.

map()

"Transform"

Transform every item. Returns a **new array** of the **exact same length**.

filter()

"Select"

Keep items passing a condition. Returns a **new array** (smaller or equal length).

reduce()

"Combine"

Aggregate items into a **single result** (number, object, string, or array).

forEach() — Concept & Syntax

Purpose

Executes a provided callback function once for each array element.

Callback Parameters

- **element:** Current item being processed.
- **index** *(optional)*: Current array position.
- **array** *(optional)*: The target array itself.

STANDARD SYNTAX

```
array.forEach((element, index, arr) => { // Side-effect code executed here });
```

Basic & Arrow Function Syntax

Standard Callback Function

```
const fruits = ["apple", "banana", "mango", "grapes"]; fruits.forEach(function (fruit) { console.log(fruit);
```

Concise Arrow Function equivalent

```
fruits.forEach(fruit ⇒ console.log(fruit));
```

FOREACH PARAMETERS

Accessing Element, Index, and Array

```
const fruits = ["apple", "banana", "mango", "grapes"]; fruits.forEach((fruit, index, arr) => { console.log(`${index}: ${fruit}`); })
```

› Console Output:

```
0: apple (Total items: 4)
1: banana (Total items: 4)
2: mango (Total items: 4)
3: grapes (Total items: 4)
```

⚠ CRITICAL PITFALL

Interview Alert: `forEach` Returns Undefined!

```
const fruits = ["apple", "banana", "mango", "grapes"]; let result = fruits.forEach(fruit => fruit.toUpperCase()); console
```



KEY TAKEAWAY

Use `forEach` strictly for side effects (logging, DOM operations). It produces no return array.

2 map() — Concept & Immutability



1-to-1 Transformation

Creates a **brand-new array** by populating it with the results of calling a function on every element. Returned array length always equals original array length.



Immutability (React Pillar)

Original array remains completely untouched! Immutability prevents side-effects and is foundational in frameworks like React and Redux.

MAP EXAMPLE

Transforming Data (Calculating Tax)

```
const prices = [100, 200, 300, 400]; const withTax = prices.map(price => price * 1.15); console.log(withTax); // [115,
```

✓ Notice how **prices** remains [100, 200, 300, 400]. The transformation occurs purely in the newly constructed array **withTax**.

Transforming Objects (Student Grades)

```
const students = [ { name: "Ali", marks: 85 }, { name: "Sara", marks: 92 }, { name: "Zain", marks: 74 } ]; // 1. Extrac
```



KEY TAKEAWAY

map transforms every element 1-to-1,
returning a brand new array while
respecting immutability.

3 filter() — Concept & Mechanics



Conditional Selection

Tests each element against a callback test. If callback returns **true**, element is kept; if **false**, element is skipped.



Output Array Dimensions

Returns a **new array** containing subset items. Result length is always **less than or equal** to original array length (0 to N).

FILTER EXAMPLE

Filtering Numeric Arrays (Even Numbers)

```
const numbers = [10, 15, 20, 25, 30, 35, 40]; // Select numbers divisible by 2 const evenNumbers = numbers.filter(num =>
```

Original: [10, 15, 20, 25, 30, 35, 40]



Filtered: [10, 20, 30, 40]

REAL-WORLD FILTER

E-Commerce Product Catalog Filters

```
const products = [ { name: "Laptop", price: 80000, inStock: true }, { name: "Mouse", price: 1500, inStock: false }, { n
```

ADVANCED TECHNIQUE

Method Chaining: Combining map() + filter()

```
// Goal: Extract names of products costing less than 30,000 const cheapProductNames = products .filter(product => produ
```

STEP 1: FILTER

Discards items $\geq 30,000$

STEP 2: MAP

Extracts name string array



KEY TAKEAWAY

Use `filter` to select elements based on boolean conditions, and chain with `map` for complex queries.

4 `reduce()` — Concept & Syntax

1. **accumulator (acc)**

Carries the accumulated running result forward between iterations.

2. **currentValue (curr)**

The current array element being evaluated.

3. **initialValue**

Starting value assigned to accumulator on

SYNTAX ANATOMY

```
array.reduce((accumulator, currentValue) => { // Return updated accumulator value return accumulator
```

REDUCE EXECUTION

Step-by-Step Execution Trace

```
const numbers = [10, 20, 30, 40]; const total = numbers.reduce((acc, curr) => { return acc + curr; }, 0); console.log("Total: ", total);
```

Calculating Shopping Cart Total

```
const cart = [ { item: "Shirt", price: 1500, qty: 2 }, { item: "Shoes", price: 4000, qty: 1 }, { item: "Cap", price: 500 }
```

REDUCE APPLICATIONS

Finding Maximum / Extremities

```
const marks = [65, 89, 72, 95, 58]; const highest = marks.reduce((acc, current) => { return current > acc ? current : acc; }, marks[0]);
```

 **Pro Tip:** Notice `marks[0]` is used as initial value. The accumulator retains the highest value encountered across every comparison.

Grouping Data into an Object

```
const fruits = ["apple", "banana", "avocado", "blueberry", "cherry"]; const grouped = fruits.reduce((acc, fruit) => { c
```



KEY TAKEAWAY

reduce condenses an array into any custom shape (number, grouped object, accumulated list) via an initial accumulator.

Quick Comparison Reference

Method	Returns	Primary Use Case
forEach()	undefined	Execute side-effects (logging, DOM rendering). No array returned.
map()	New Array (Same length)	Transform every item into a new representation 1-to-1.
filter()	New Array (≤ length)	Select items matching condition criteria.
reduce()	Single Value / Structure	Aggregate values into sums, averages, totals, or grouped objects.

HOMWORK TASK



Homework Assignment

```
const employees = [ { name: "Ahmed", salary: 45000, department: "IT" }, { name: "Fatima", salary: 60000, department: "H
```

RECAP

Session Recap

- ✓ **forEach:** Side-effect iteration loop. Never returns a value (**undefined**).
- ✓ **map:** Produces a brand new transformed array of equal length without mutating original data.
- ✓ **filter:** Constructs a subset array based on boolean evaluation conditions.
- ✓ **reduce:** Aggregates entire array into a single result (numbers, objects, or arrays).

UP NEXT • DAY 08

JavaScript Objects & **this** Keyword

Object basics, object methods, destructuring, and unravelling the mechanics of the **this** keyword.

♥ Keep Coding & See You Next Class!